Universidad de Granada

Periféricos y Dispositivos de Interfaz Humana
Curso 2025/2026

Proyecto. Código Morse usando Arduino

Autora:
Inés Ruiz Sánchez

21 de abril de 2026

UNIVERSIDAD
DE GRANADA

Índice

1. Introducción **2**
1.1. ¿Qué es el código Morse? . . . 2
1.2. Objetivo del proyecto . . . 2

2. Diseño hardware **2**
2.1. Listado de componentes . . . 2
2.2. Esquema de conexión . . . 3

3. Desarrollo del software **5**
3.1. Módulo Transmisor (Texto a Morse) . . . 5
3.2. Módulo Receptor (Morse a Texto) . . . 8

4. Implementación física **8**

5. Conclusiones, problemas y posibles Mejoras **8**

6. Anexo **8**
6.1. Listado de componentes . . . 8
6.1.1. Transmisor - Versión 1: Señal luminosa . . . 8
6.1.2. Transmisor - Versión 2: Señal luminosa y acústica . . . 8
6.1.3. Receptor . . . 8
6.2. Código completo del Módulo Transmisor (texto a Morse) . . . 8
6.2.1. Versión 1. Señal luminosa . . . 8
6.2.2. Versión 2. Señal luminosa y acústica . . . 10
6.3. Código completo del Módulo Receptor (Morse a texto) . . . 11
6.3.1. Versión 1. . . . 11
6.3.2. Versión 2. . . . 11

Índice de figuras

1. Circuito del módulo Transmisor en Tinkercad . . . 3
2. Vista esquemática del Traductor . . . 4
3. Circuito del módulo Transmisor en Tinkercad. Versión 2 . . . 5

1. Introducción

1.1. ¿Qué es el código Morse?

- La duración de una raya es tres veces la de un punto
- El tiempo que transcurre entre cada raya o punto es igual a la duración de un punto
- El espacio entre dos letras tiene la misma longitud que la raya
- El espacio entre dos palabras tiene la misma duración que siete puntos

Letra	Código	Letra	Código	Número	Código
A	.–	N	–.	1	. – – – –
B	–...	O	– – –	2	.. – ––
C	–. – .	P	. – –.	3	... – –
D	–..	Q	– – .–	4	....–
E	.	R	. – .	5	.....
F	.. – .	S	...	6	–....
G	– – .	T	–	7	– – ...
H	....	U	..–	8	– – –..
I	..	V	...–	9	– – – – .
J	. – ––	W	. – –	0	– – – – –
K	–.–	X	–..–		
L	. – ..	Y	–. – –		
M	––	Z	– – ..		

Cuadro 1: Alfabeto internacional y números en Código Morse

1.2. Objetivo del proyecto

- **Codificador** - Modo transmisor. Programar el Arduino para que, dada una cadena de texto (por ejemplo, una palabra escrita en el código o enviada desde el PC por el puerto serie), el Arduino la traduzca y la emita haciendo parpadear un LED y sonando un zumbador (buzzer) con los tiempos exactos del código Morse (punto corto, raya larga).
- **Decodificador** - Modo Receptor. Conectar un pulsador (botón) al Arduino. El usuario hará pulsaciones cortas y largas. El programa deberá medir el tiempo que el botón está pulsado para distinguir entre "puntosz rayas", agruparlos, y mostrar por el monitor serie (o en una pequeña pantalla LCD) la letra o palabra que se está tecleando.

2. Diseño hardware

2.1. Listado de componentes

Para hacer un **transmisor de código Morse** se enviará un mensaje desde el ordenador a la placa Arduino a través del cable USB. Por tanto los **componentes eléctricos** necesarios son:

- Placa Arduino UNO
- Placa de pruebas (*protoboard*)
- Resistencia de 270Ω
- LED en el pin 12
- Buzzer (altavoz piezoeléctrico)

- Cables para conectar los componentes
- Cable USB para conectar placa Arduino con el ordenador

En el anexo [6.1] se puede ver una tabla que especifica los componentes del *Transmisor* para cada versión.

Para la **primera versión**, en la que solo hay una señal luminosa (LED) la tabla correspondiente es: Cuadro [2].

En una **segunda versión** añadimios el buzzer para poder añadir una señal acústica (Cuadro [3]).

Los **componentes eléctricos** utilizados para el **receptor de código Morse** son:

- Placa Arduino UNO
- Placa de pruebas (*protoboard*)
- Cables para conectar los componentes
- Cable USB para conectar placa Arduino con el ordenador

2.2. Esquema de conexión

Circuito del módulo **Transmisor**:

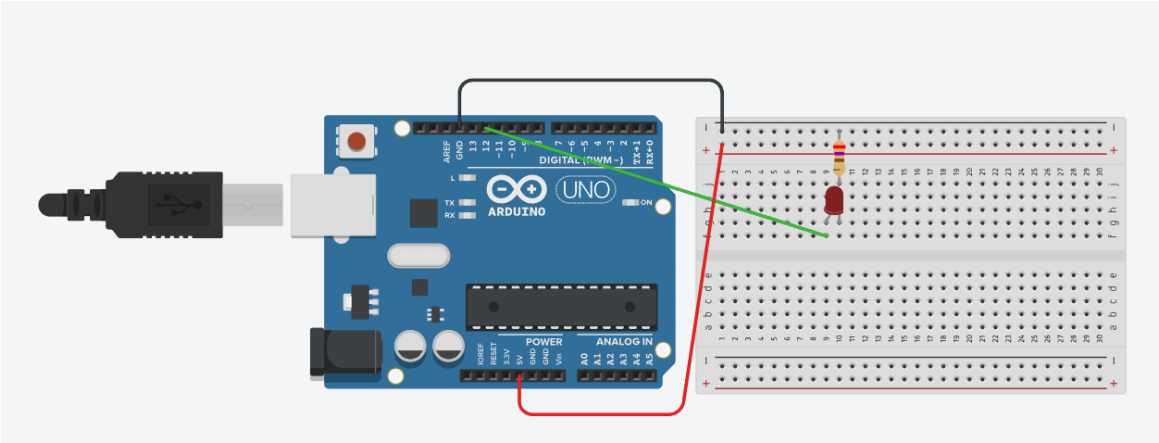

Figura 1: Circuito del módulo Transmisor en Tinkercad

Se puede observar que el LED está conectado al pin 12 del Arduino.

Link al circuito Transmisor en Tinkercad.
Vista esquemática del circuito para el *Transmisor*:

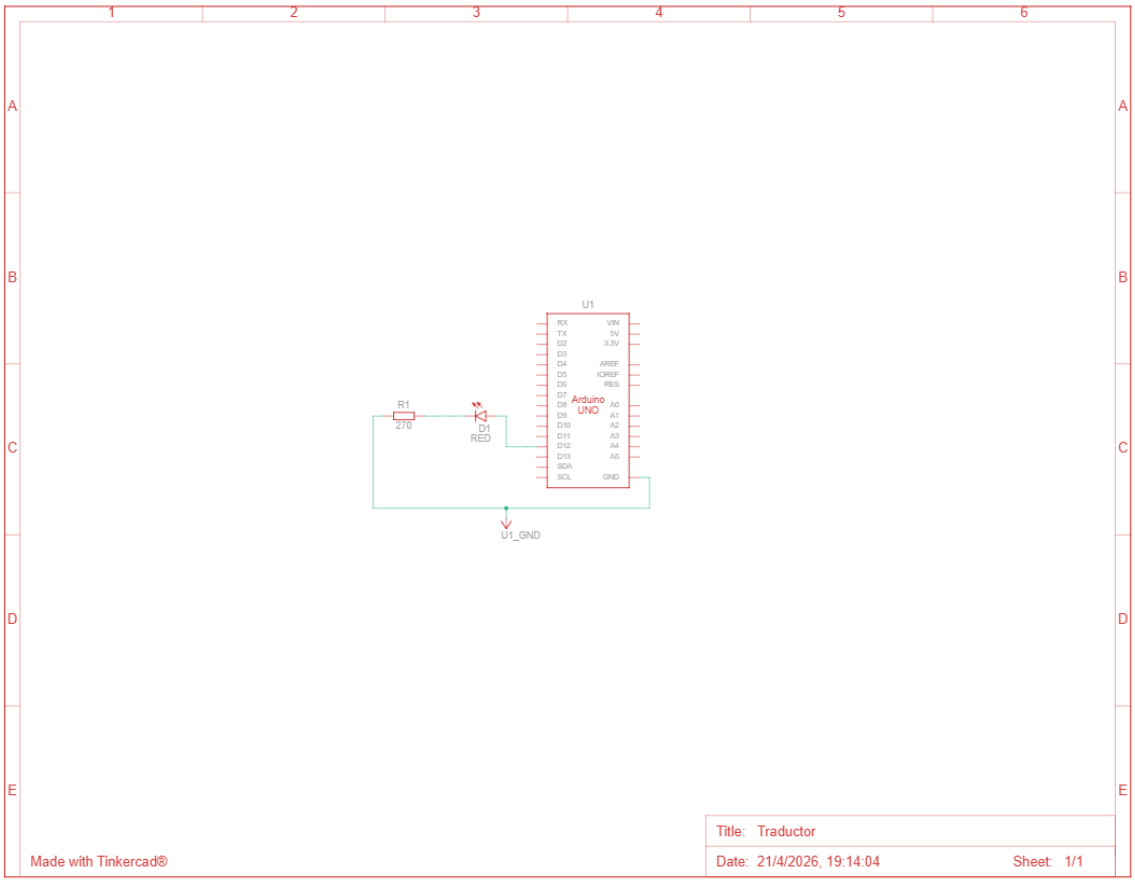

Figura 2: Vista esquemática del Traductor

Para la segunda versión, en la que se añade un buzzer, se conecta su terminal positivo al pin 11 del Arduino.

images/02_circuitoTransmisor2.png

Figura 3: Circuito del módulo Transmisor en Tinkercad. Versión 2

3. Desarrollo del software

3.1. Módulo Transmisor (Texto a Morse)

Permite escribir una frase en nuestro ordenador y hacer que la placa Arduino lo convierta en los puntos y rayas apropiados del código Morse encendiendo y apagando un LED.

Para la comunicación con el Arduino uso tres funciones:

- *Serial.begin(9600)* inicializa la comunicación entre el ordenador y la placa a una velocidad de 9600 baudios.
- La función *Serial.available()* comprueba constantemente el búfer de entrada del microcontrolador. Si devuelve un valor mayor que 0, significa que el usuario ha tecleado algo en el Monitor Serie del ordenador y el Arduino está listo para procesarlo.
- *Serial.read()* extrae el primer carácter disponible en el búfer y lo almacena en la variable caracter para su evaluación.

En vez de comprobar caracter a caracter, he creado unos vectores de caracteres para almacenar las letras del abecedario y los números utilizados.

```
// Array con las letras del alfabeto Morse (De la 'A' a la 'Z')
char* letras[] = {
  ".-",    "-...", "-.-.", "-..",  ".",    // A, B, C, D, E
  "..-.", "--.",  "....", "..",   ".---",// F, G, H, I, J
  "-.-",  ".-..", "--",   "-.",   "---", // K, L, M, N, O
  ".--.", "--.-", ".-.",  "...",  "-",   // P, Q, R, S, T
  "..-",  "...-", ".--",  "-..-", "-.--",// U, V, W, X, Y
  "--.."  // Z
};

// Array con los números del alfabeto Morse (Del '0' al '9')
char* numeros[] = {
  "-----", ".----",  // 0, 1
  "..---", "...--",  // 2, 3
  "....-", ".....",  // 4, 5
  "-....", "--...",  // 6, 7
  "---..", "----."   // 8, 9
};
```

Listing 1: Vectores de caracteres

La función *loop* se encarga de comprobar si hay datos para leer del USB y mostrarlos.

```
void loop(){
  char caracter;

  // Comprobación de datos a leer
  if(Serial.available()){ // Si hay algo que leer del USB
    caracter = Serial.read(); // Leer una letra

    // Según el caracter
    if(caracter >= 'a' && caracter <= 'z'){
        mostrarSecuencia(letras[caracter - 'a']); // Minúscula
    }    else if(caracter >= 'A' && caracter <= 'Z'){
        mostrarSecuencia(letras[caracter - 'A']); // Mayúscula
    }else if(caracter >= '0' && caracter <= '9'){
        mostrarSecuencia(numeros[caracter - '0']); // Número
    }else if(caracter == '␣'){
        delay(espera *4); // Espacio entre palabras
    }
  }
}
```

Listing 2: Función loop de Transmisor

Cuando en el código se hace $letras[caracter - 'a']$ el programa resta el valor ASCII de la letra 'a' minúscula al carácter introducido. Si el usuario teclea una 'c', la operación matemática subyacente es 99 ('c') - 97 ('a') = 2. Automáticamente, el programa busca en la posición 2 del array *letras*, que corresponde exactamente al código Morse de la 'C' ($-.-.$). Esto mismo se aplica para mayúsculas y números.

La variable $espera = 200$ (en milisegundos) que representa la duración de un "punto". A partir de esta constante, se calculan todos los demás tiempos estándar del código Morse internacional de forma proporcional:

- Un punto (.) dura 1 unidad de *espera* (*delay(espera)*).
- Una raya ($-$) dura 3 unidades (*delay(espera * 3)*).
- El espacio entre partes de la misma letra es de 1 unidad.
- El espacio entre letras es de 3 unidades (*delay(espera * 3)*).
- El espacio entre palabras diferentes (cuando se lee un espacio en blanco ' ') es de 4 unidades.

Además, he implementado dos funciones auxiliares para dar modularidad al código.

- *mostrarSecuencia (char* secuencia)* que recorre y emite los caracteres de la secuencia que se pasa por parámetro. Recorrerá la cadena hasta llegar al caracter nulo de fin de cadena.

```
void mostrarSecuencia(char* secuencia){
    int i=0;

    // Recorre la cadena hasta encontrar el carácter nulo de fin de cadena (NULL
        )
    while(secuencia[i] != NULL){
        encenderLED(secuencia[i]); // Emite punto o raya
        i++;
    }
    delay(espera *3); // Espacio entre letras
}
```

Listing 3: Función loop de Transmisor

- *encenderLED (char puntoOBarra)* sirve para encender el LED físico. Hace uso de las funciones *digitalWrite()* para encender con el parámetro *HIGH* y apagar con el parametro *LOW*.

```
void encenderLED(char puntoOBarra){
    digitalWrite(ledPin, HIGH); // Encender LED

    if(puntoOBarra == '.'){
        delay(espera); // Pausa de un punto (1 unidad de tiempo)
    }else{ // es -
        delay(espera *3); // Pausa de un guion (3 unidades de tiempo)
    }

    digitalWrite(ledPin, LOW); // Apagar LED
    delay(espera); // espacio entre destellos
}
```

Listing 4: Función loop de Transmisor

El código completo se puede ver en el anexo **6.2.1**.

Esta primera aproximación es muy sencilla pero funcional. Tiene una limitación y es la distancia a la que se ve el LED. Si el LED está muy lejos del usuario receptor entonces no se verá. Una solución es poner un LED más potente u otro sistema lumínico. Para mejorar y añadir un poco el *Traductor* vamos a añadir un **zumbador** (buzzer). Al añadir sonido estamos involucrando otro sentido y puede ser más fácil entender el código.

Solo se necesita una constante para el *pin* del buzzer y ponerlo en *OUTPUT* en el *setup()*. En la función *mostrarSecuencia* hay que añadir un *tone(buzzPin, 1000)* justo después del *digitalWrite(ledPin, HIGH)*, y un *noTone(buzzPin)* después del *LOW*.

```
void encenderSignal(char puntoOBarra)
{
  digitalWrite(ledPin, HIGH); // Encender LED
  tone(buzzPin, 1000); // Activar zumbador

  if(puntoOBarra == '.')
  {
    delay(espera); // Pausa de un punto (1 unidad de tiempo)
  }
  else // es -
  {
    delay(espera *3); // Pausa de un guion (3 unidades de tiempo)
  }

  digitalWrite(ledPin, LOW); // Apagar LED
  noTone(buzzPin); // Desactivar zumbador

```

```
  delay(espera); // espacio entre destellos
}
```

Listing 5: Actualización Transmisor

El código completo se puede ver en el anexo [6.2.2].

3.2. Módulo Receptor (Morse a Texto)

4. Implementación física

5. Conclusiones, problemas y posibles Mejoras

6. Anexo

6.1. Listado de componentes

6.1.1. Transmisor - Versión 1: Señal luminosa

Cuadro de componentes eléctricos necesarios para el circuito más sencillo: solo una señal luminosa.

Componente	Cantidad
Arduino UNO R3	1
Resistencia de 270Ω	1
LED	1

Cuadro 2: Componentes Transmisor versión 1

6.1.2. Transmisor - Versión 2: Señal luminosa y acústica

Cuadro de componentes eléctricos para traducir el texto a señal luminosa y acústica.

Componente	Cantidad
Arduino UNO R3	1
Resistencia de 270Ω	1
Buzzer	1
LED	1

Cuadro 3: Componentes Transmisor versión 2

6.1.3. Receptor

**Modificar tabla.

Componente	Cantidad
Arduino UNO R3	1

Cuadro 4: Componentes Receptor

6.2. Código completo del Módulo Transmisor (texto a Morse)

6.2.1. Versión 1. Señal luminosa

```
// Constantes y vectores
int ledPin = 12; // Constante para el pin del LED
int espera = 200; // Duración de un punto
```

```

// Array con las letras del alfabeto Morse (De la 'A' a la 'Z')
char* letras[] = {
  ".-",   "-...", "-.-.", "-..",  ".",   // A, B, C, D, E
  "..-.", "--.",  "....", "..",   ".---",// F, G, H, I, J
  "-.-",  ".-..", "--",   "-.",   "---", // K, L, M, N, O
  ".--.", "--.-", ".-.",  "...",  "-",   // P, Q, R, S, T
  "..-",  "...-", ".--",  "-..-", "-.--",// U, V, W, X, Y
  "--.."  // Z
};

// Array con los números del alfabeto Morse (Del '0' al '9')
char* numeros[] = {
  "-----", ".----",  // 0, 1
  "..---", "...--",  // 2, 3
  "....-", ".....",  // 4, 5
  "-....", "--...",  // 6, 7
  "---..", "----."   // 8, 9
};

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600); // Inicia la comunicación
}

void loop()
{
  char caracter;

  // Comprobación de datos a leer
  if(Serial.available()){ // Si hay algo que leer del USB
    caracter = Serial.read(); // Leer una letra

    // Según el caracter
    if(caracter >= 'a' && caracter <= 'z'){
        mostrarSecuencia(letras[caracter - 'a']); // Minúscula
    }
    else if(caracter >= 'A' && caracter <= 'Z'){
        mostrarSecuencia(letras[caracter - 'A']); // Mayúscula
    }
    else if(caracter >= '0' && caracter <= '9'){
        mostrarSecuencia(numeros[caracter - '0']); // Número
    }
    else if(caracter == '␣'){
        delay(espera *4); // Espacio entre palabras
    }
  }
}

// Función auxiliar para emitir simboloes de un caracter
void mostrarSecuencia(char* secuencia)
{
  int i=0;

  // Recorre la cadena hasta encontrar el carácter nulo de fin de cadena (NULL)
  while(secuencia[i] != NULL)
  {
    encenderLED(secuencia[i]); // Emite punto o raya
    i++;
  }
  delay(espera *3); // Espacio entre letras
}
```

```

// Función auxiliar para encendder el LED
void encenderLED(char puntoOBarra)
{
  digitalWrite(ledPin, HIGH); // Encender LED

  if(puntoOBarra == '.')
  {
    delay(espera); // Pausa de un punto (1 unidad de tiempo)
  }
  else // es -
  {
    delay(espera *3); // Pausa de un guion (3 unidades de tiempo)
  }
  digitalWrite(ledPin, LOW); // Apagar LED
  delay(espera); // espacio entre destellos
}
```

6.2.2. Versión 2. Señal luminosa y acústica

```
// Constantes y vectores
int ledPin = 12; // Constante para el pin del LED
int buzzPin = 11; // Constante para el pin del zumbador
int espera = 200; // Duración de un punto

// Array con las letras del alfabeto Morse (De la 'A' a la 'Z')
char* letras[] = {
  ".-",   "-...", "-.-.", "-..",  ".",   // A, B, C, D, E
  "..-.", "--.",  "....", "..",   ".---",// F, G, H, I, J
  "-.-",  ".-..", "--",   "-.",   "---", // K, L, M, N, O
  ".--.", "--.-", ".-.",  "...",  "-",   // P, Q, R, S, T
  "..-",  "...-", ".--",  "-..-", "-.--",// U, V, W, X, Y
  "--.."  // Z
};

// Array con los números del alfabeto Morse (Del '0' al '9')
char* numeros[] = {
  "-----", ".----",  // 0, 1
  "..---", "...--",  // 2, 3
  "....-", ".....",  // 4, 5
  "-....", "--...",  // 6, 7
  "---..", "----."   // 8, 9
};

void setup()
{
  pinMode(ledPin, OUTPUT);
  pinMode(buzzPin, OUTPUT);
  Serial.begin(9600); // Inicia la comunicación
}

void loop()
{
  char caracter;

  // Comprobación de datos a leer
  if(Serial.available()){ // Si hay algo que leer del USB
    caracter = Serial.read(); // Leer una letra

    // Según el caracter
    if(caracter >= 'a' && caracter <= 'z'){
        mostrarSecuencia(letras[caracter - 'a']); // Minúscula
```

```
    }
    else if(caracter >= 'A' && caracter <= 'Z'){
        mostrarSecuencia(letras[caracter - 'A']); // Mayúscula
    }
    else if(caracter >= '0' && caracter <= '9'){
        mostrarSecuencia(numeros[caracter - '0']); // Número
    }
    else if(caracter == '␣'){
        delay(espera *4); // Espacio entre palabras
    }
  }
}

// Función auxiliar para emitir simboloes de un caracter
void mostrarSecuencia(char* secuencia)
{
  int i=0;

  // Recorre la cadena hasta encontrar el carácter nulo de fin de cadena (NULL)
  while(secuencia[i] != NULL)
  {
    encenderLED(secuencia[i]); // Emite punto o raya
    i++;
  }
  delay(espera *3); // Espacio entre letras
}

// Función auxiliar para encender el LED y buzzer
void encenderSignal(char puntoOBarra)
{
  digitalWrite(ledPin, HIGH); // Encender LED
  tone(buzzPin, 1000); // Activar zumbador

  if(puntoOBarra == '.')
  {
    delay(espera); // Pausa de un punto (1 unidad de tiempo)
  }
  else // es -
  {
    delay(espera *3); // Pausa de un guion (3 unidades de tiempo)
  }

  digitalWrite(ledPin, LOW); // Apagar LED
  noTone(buzzPin); // Desactivar zumbador

  delay(espera); // espacio entre destellos
}
```

6.3. Código completo del Módulo Receptor (Morse a texto)

6.3.1. Versión 1.

6.3.2. Versión 2.

Referencias

[1] E. Schernich y P. Martínez. *Arduino para principiantes: aprendizaje mediante programación por bloques para docentes y toda la familia*. 2a ed. Barcelona: Marcombo, 2019.

[2] S. Monk y J. Pompa. *30 proyectos con Arduino*. Madrid: Estribor, 2012.